

4. Phase 2 ADS PSV Semantic Binding: Environment, Conversion Operations, and Function-Level Processing

This document covers **src/semantic_binding.py**, which converts Phase 1 Structured CSV into ADS PSV. ISO 21378 ADC CSV views use the same flat-file conversion engine and are retained as an appendix here.

ADS XBRL GL is a separate XML syntax-binding process and is documented in [05_PHASE2_ADS_XBRL_GL_SYNTAX_BINDING.md](#).

Processing Flow

```
Structured CSV + semantic binding CSV
→ register repeated classes from C rows
→ derive source columns from A-row semantic_path
→ resolve [n] and row scope
→ merge by invoice or repeated class occurrence
→ write binding-order PSV/CSV
```

ADS PSV and Flat-File Contract

1. Purpose

This document specifies the semantic binding conversion program used by the UADC Proof of Concept.

The program converts a Phase 1 UADC Structured CSV into a Phase 2 target flat file by applying a semantic binding CSV. The target flat file can be pipe-separated PSV or comma-separated CSV. The ADS PSV files and ISO 21378 ADC invoice CSV files use the same processing model.

Implementation-level processing details are included in Chapters 7, 8, and 15 of this document.

All paths in this document are relative to the **UADC_PoC** working directory.

2. Main Program

Program:

```
src/semantic_binding.py
```

The converter is independent of XML syntax. It reads Structured CSV rows and maps values to target columns by **semantic_path**.

3. Input Files

3.1 Structured CSV input

The input is a Phase 1 Structured CSV file generated by **src/syntax_binding.py**.

Example:

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
```

The input may also be a directory containing multiple Structured CSV files. In that case, each **.csv** file is converted separately.

3.2 Semantic binding CSV

Semantic binding CSV files are under:

```
specs/bindings/semantic/
```

Current ADS files include:

```
ADS_Invoices_Received_PSV_Binding.csv
ADS_Invoices_Generated_PSV_Binding.csv
ADS_Invoices_Received_Lines_PSV_Binding.csv
ADS_Invoices_Generated_Lines_PSV_Binding.csv
ADS_Supplier_Listing_PSV_Binding.csv
ADS_Customer_Master_PSV_Binding.csv
```

Current ISO 21378:2019 ADC invoice files include:

```
ISO21378_SAL_Invoice_Generated_CSV_Binding.csv
ISO21378_SAL_Invoice_Generated_Details_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv
```

They implement the flat invoice views from Tables 38, 39, 53, and 54. Repeated TAX groups are expanded to **Tax1** through **Tax4** columns. Repeated BUSINESS SEGMENT values are represented by **Business_Segment_01** through **Business_Segment_05** columns. This physical expansion is needed because a CSV header cannot contain repeated field names.

4. Binding Table Contract

The semantic binding table starts from the target definition table and adds UADC mapping columns:

```
field_no,field_name,level,flat_file_data_type,length,description,source_document,semantic_path,type,multiplicity,mapping_status,mapping_note
```

4.1 A rows

Rows with **type=A** emit target columns.

Important fields:

- **field_no** controls output column order.
- **field_name** is the emitted target column name.
- **semantic_path** identifies the UADC Structured CSV value.
- **description** comes from the target definition document.
- **mapping_status** is **direct**, **approximate**, **requires_transformation**, or **not_available**.
- **mapping_note** explains a semantic approximation or data gap.

The converter derives the Structured CSV source column from the final segment of **semantic_path**.

Example:

```
$.invoice.invoiceNumber -> InvoiceNumber  
$.invoice.seller.sellerIdentifier -> SellerIdentifier
```

4.2 C rows

Rows with **type=C** define UADC semantic classes used for row-scope resolution.

Important fields:

- **field_name** is the semantic class name.
- **description** is copied from the Structured CSV model definition.
- **semantic_path** identifies the semantic class.
- **multiplicity** determines whether the class can repeat.

Repeated classes are those whose multiplicity upper bound is *****, **n**, **unbounded**, or greater than 1.

The converter derives the Structured CSV dimension column from the class semantic path.

Example:

```
$.invoice.invoiceLine -> dInvoiceLine
```

5. Repeated Row Scope

The converter determines the target row scope from the deepest repeated **C** ancestor used by non-indexed **A** rows.

Invoice-level targets have no repeated row scope. The converter merges sparse Structured CSV rows into one target row per invoice.

Line-level targets use **\$.invoice.invoiceLine** as row scope. The converter emits one target row for each **dInvoiceLine** occurrence and merges parent invoice values into each line row.

Other repeated classes, such as VAT breakdown or allowances, follow the same rule when a semantic binding table uses those classes as non-indexed repeated ancestors.

6. Indexed Repeated Values

Use zero-based indexed semantic paths when a repeated group must be expanded horizontally into target columns.

Example:

```
$.invoice.vatBreakdown[0].vatCategoryCode  
$.invoice.vatBreakdown[1].vatCategoryCode  
$.invoice.vatBreakdown[2].vatCategoryCode
```

Indexed paths do not create additional output rows. They select the specified occurrence inside the current invoice or current repeated row context.

The ISO 21378 header bindings use indexed VAT breakdown paths for **Tax1** through **Tax4**. Invoice notes and payment instructions also use occurrence zero because their ADC header fields are singular. The detail bindings use the repeated **InvoiceLine** class as the output row scope. EN 16931 permits one VAT classification per invoice line, so only **Tax1_Type_Code** can be mapped directly on detail rows.

7. Internal Data Structures

read_csv_rows(path) returns:

```
list[dict[str, str]]
```

Each dictionary is one CSV row keyed by header name.

SemanticClass stores:

- **semantic_path**;
- **multiplicity**.

Repeated **SemanticClass** rows are collected as:

```
dict[str, SemanticClass]
```

SemanticBinding stores one target-column binding:

- **sequence**;
- **target_column**;
- **semantic_path**;
- **normalized_semantic_path**;
- **source_column**;
- **repeat_group_path**;
- **repeat_group_column**;
- **repeat_index**;
- **default_value**;
- **required**.

load_bindings(binding_csv) returns:

```
list[SemanticBinding]
```

The list is sorted by **field_no** or **sequence**, then by target column name.

8. Processing Functions

load_bindings(binding_csv) reads the binding table, records repeated **C** classes, creates **SemanticBinding** objects for **A** rows, derives source columns from semantic paths, and resolves repeated row scopes from the binding table.

row_scope_group(bindings) selects the deepest repeated group used by the target view. This determines whether the output is invoice-level or repeated-row-level.

transform_rows(rows, bindings) applies the binding table to Structured CSV rows. If no repeated row scope exists, it merges sparse rows into invoice-level target rows.

transform_repeated_group_rows(rows, bindings, scope_path, scope_column) emits one target row per repeated group occurrence, such as one row per **dInvoiceLine**.

row_repeat_indices(source_row, bindings, repeat_counts) assigns zero-based occurrence numbers for indexed repeated paths.

merge_values(target_row, source_row, bindings, repeat_indices) copies non-empty Structured CSV values into target columns. It uses the first non-empty value for each target column.

output_path(input_csv, output_dir, extension, output_filename, binding_csv) writes output under:

```
out/phase2/<target-family>/<structured-csv-stem>/<target-view>.<extension>
```

unless **--output-filename** is supplied for a single input file.

9. Output Contract

The output header follows the order of **A** rows in the semantic binding CSV.

Default output presets:

Format	Delimiter	Extension
psv	**	**
csv	,	.csv

The target view stem is derived from the binding file name. For example:

```
ADS_Invoices_Received_PSV_Binding.csv -> Invoices_Received.psv
ADS_Customer_Master_PSV_Binding.csv -> Customer_Master.psv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv -> PUR_Invoice_Received.csv
```

10. Non-Goals

- The converter does not parse XML.
- The converter does not use XPath.
- The converter does not infer mappings from target column names alone.
- The converter does not read an external LHM CSV at runtime.
- The converter does not generate xBRL-CSV metadata JSON.
- The converter does not derive fiscal year or accounting period without an accounting calendar.
- The converter does not calculate line gross amounts or line tax amounts.
- The converter does not invent ERP audit trail, posting account, status, or additional business segment values that are absent from EN 16931.

11. Operational Workflow

Phase 2 reads the Phase 1 Structured CSV and a target semantic-binding table. Select the binding for the required target view, run the corresponding converter, and inspect the generated file under

out/phase2/. All mappings are resolved from **field_name**, **semantic_path**, class **multiplicity**, and binding order.

The binding's **type=C** rows establish class scopes. **type=A** rows define target fields. The converter derives the Structured CSV source column from the final semantic-path segment and assigns each field to its deepest repeated class ancestor.

12. ADS PSV

ADS PSV output is produced by **src/semantic_binding.py** with the **psv** preset. The six ADS binding views cover invoice and master-data perspectives required by the proof of concept. Each output header follows binding order, and repeated target records follow the repeated semantic class selected by the binding.

Fields that depend on ERP user history, posting status, journal accounts, or period-close processing remain empty when EN 16931 supplies no source value. This is a declared source-data boundary, not an inferred zero or default.

14. ISO 21378 ADC

The ISO 21378 ADC implementation uses four invoice semantic-binding tables and the CSV preset of **src/semantic_binding.py**. It demonstrates that the same Structured CSV can feed an independently designed accounting-data interface.

Coverage is limited to fields supported by the invoice. Transaction approval, user action history, posting details, closing information, and report-production state require ERP sources. The generated ADC file therefore preserves known invoice values and explicit gaps without inventing accounting evidence.

15. Function-Level Processing Reference

15.1 Delimited ADS and ADC Conversion

- **load_bindings** reads class and attribute rows in sequence and validates required target fields.
- **indexed_semantic_path** recognizes explicitly indexed repeated values.
- **source_column_for_path** maps the final semantic-path segment to the Structured CSV column.
- **dimension_column_for_path** identifies the dimension associated with a class path.
- **deepest_repeated_class_ancestor** and **resolve_bindings_for_classes** assign each target field to its effective repeated row scope.
- **transform_rows** maintains current ancestor values and creates target rows.
- **transform_repeated_group_rows** emits independently indexed repeated records.
- **merge_values** copies only values owned by the current row and required by the target binding.
- **write_target_file** writes the ordered PSV or CSV header and records.

A singular child class is merged into the current repeated ancestor. A repeated child creates independent target records from its first occurrence, matching the Structured CSV contract in

02_STRUCTURED_CSV_LHM_BINDINGS.md.

16. Validation and Troubleshooting

Validate all target files for all Phase 1 examples, not only one representative invoice. For ADS XBRL GL, run Arelle validation and confirm that every instance loads its schema and relationships. For ADS PSV and ADC, confirm header order, record count, delimiter, and representative repeated values.

If a field is absent, check in this order:

1. the value exists in the Phase 1 Structured CSV;
2. the semantic path ends in the expected source column;
3. all **type=C** ancestors and multiplicities exist in the binding;
4. selector context is retained by the join table;
5. the target path and target field name are populated;
6. the row belongs to the repeated scope expected by the target view.

17. Semantic Binding Conversion Overview

Semantic Binding Conversion Documentation

This directory documents the Structured-CSV-to-target-flat-file conversion program.

The semantic binding converter reads a Phase 1 UADC Structured CSV and a semantic binding CSV. It writes a Phase 2 target flat file such as ADS PSV or ISO 21378 ADC CSV. The binding table defines which target column receives which UADC **semantic_path** value, and which semantic class controls repeated target rows.

The ISO 21378:2019 ADC invoice binding currently covers Tables 38, 39, 53, and 54. The target TAX groups are expanded to four numbered column sets and BUSINESS SEGMENT is expanded to five numbered columns so that every emitted CSV header is unique. Each target row also records **mapping_status** and **mapping_note**. These columns distinguish direct mappings from semantic approximations, required transformations, and source data that is not available from an EN 16931 invoice.

These four views complete the ISO 21378 part of the planned Phase 2 PoC baseline. Completion means that target definitions, bindings, CSV generation, regression execution, and explicit gap classification are present. It does not assert full ISO 21378 data completeness from an EN 16931 source invoice.

For implementation-level details shared with the broader processing model, including internal **dict/list/dataclass** objects and function-level data flow, use **02_STRUCTURED_CSV_LHM_BINDINGS.md**, Chapter 15.

Files

- **program_specification.md** - Defines converter inputs, outputs, semantic binding rows, repeated row scope, indexed repeated values, output naming, and non-goals.

- **user_guide.md** - Gives command examples for ADS PSV and CSV generation, directory input, and troubleshooting.
- **iso21378_adc_invoice_coverage.md** - Records ISO 21378 ADC invoice mapping coverage, data gaps, and the Phase 1 regression result.

Related Directories

- **../src/** - Converter implementation, especially **semantic_binding.py**.
- **../specs/bindings/semantic/** - Active semantic binding CSV files.
- **../specs/bindings/syntax/** - Syntax binding CSV files that define the Phase 1 Structured CSV model used as semantic source.
- **../out/phase1/** - Phase 1 Structured CSV inputs.
- **../out/phase2/** - Generated Phase 2 target outputs.

Phase 2 semantic binding starts from the common Structured CSV. It does not parse XML directly and does not depend on source XML XPath expressions.

18. Semantic Binding User Guide

User Guide: Semantic Binding Structured-CSV-to-Flat-File Conversion

1. Working Directory

Run commands from the **UADC_PoC** directory:

```
cd UADC_PoC
```

Set the Python command for Windows PowerShell:

```
$python = 'python'
```

For macOS or Linux shell:

```
PYTHON=python3
```

2. Main Script

Use:

```
src/semantic_binding.py
```

The script converts Phase 1 Structured CSV into Phase 2 target PSV or CSV files using a semantic binding CSV.

3. Generate ADS Invoices Received PSV

Run:

```
& $python .\src\semantic_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
-b .\specs\bindings\semantic\ADS_Invoices_Received_PSV_Binding.csv `
-o .\out\phase2\ADS_PSV `
--format psv
```

Output:

```
out/phase2/ADS_PSV/openpeppol_ubl_invoice_minimal/Invoices_Received.psv
```

4. Generate ADS Invoices Received Lines PSV

Run:

```
& $python .\src\semantic_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
-b .\specs\bindings\semantic\ADS_Invoices_Received_Lines_PSV_Binding.csv `
-o .\out\phase2\ADS_PSV `
--format psv
```

Output:

```
out/phase2/ADS_PSV/openpeppol_ubl_invoice_minimal/Invoices_Received_Lines.
psv
```

This target has a repeated row scope. The binding table includes **type=C** for **\$.invoice.invoiceLine** with repeated multiplicity, so the converter emits one row per **dInvoiceLine** occurrence.

5. Convert All Phase 1 Structured CSV Files

Run:

```
& $python .\src\semantic_binding.py `
.\out\phase1 `
-b .\specs\bindings\semantic\ADS_Invoices_Received_PSV_Binding.csv `
-o .\out\phase2\ADS_PSV `
--format psv
```

Each input CSV creates one output directory named by the input Structured CSV stem.

Example:

```
out/phase2/ADS_PSV/Allowance-example/Invoices_Received.psv
```

6. Generate CSV Instead Of PSV

Use **--format csv**:

```
& $python .\src\semantic_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
-b .\specs\bindings\semantic\ADS_Invoices_Received_PSV_Binding.csv `
-o .\out\phase2\ADS_CSV `
--format csv
```

Output:

```
out/phase2/ADS_CSV/openpeppol_ubl_invoice_minimal/Invoices_Received.csv
```

7. Generate ISO 21378 ADC Invoice CSV

Use the purchase invoice header binding for ISO 21378:2019 Table 53:

```
& $python .\src\semantic_binding.py `
.\out\phase1\bis-billing3-examples\Allowance-example.csv `
-b
.\specs\bindings\semantic\ISO21378_PUR_Invoice_Received_CSV_Binding.csv `
-o .\out\phase2\ISO21378_ADC `
--format csv
```

Output:

```
out/phase2/ISO21378_ADC/Allowance-example/PUR_Invoice_Received.csv
```

Use **ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv** for Table 54. Use the corresponding **SAL** files for Tables 38 and 39.

Review **mapping_status** in each binding before treating the output as a complete ADC submission:

- **direct** means the UADC value can be copied.
- **approximate** means the closest EN 16931 value is copied but the semantics are not identical.
- **requires_transformation** means a calculation, parsing rule, code conversion, or accounting calendar is required.
- **not_available** means the EN 16931 invoice does not contain the required ERP or audit datum.

8. Command Options

Basic form:

```
& $python .\src\semantic_binding.py INPUT `
-b BINDING_CSV `
-o OUTPUT_DIR `
--format psv
```

Options:

- **INPUT**: one Structured CSV file or a directory containing Structured CSV files.
- **-b, --binding-csv**: semantic binding CSV.
- **-o, --output-dir**: output directory.
- **--format**: output preset, either **psv** or **csv**.
- **--delimiter**: explicit delimiter. This overrides **--format**.
- **--extension**: explicit output extension. This overrides **--format**.
- **--output-filename**: explicit output filename for single-file input.

9. Binding Table Review Points

For target fields:

- Use **type=A**.
- Put the emitted target column name in **field_name**.
- Put the UADC source in **semantic_path**.
- Keep **field_no** in the target output order.

For row-scope classes:

- Use **type=C**.
- Put the semantic class path in **semantic_path**.

- Put the Structured CSV model description in **description**.
- Put the class multiplicity in **multiplicity**.

For horizontally repeated values:

- Use zero-based indexed paths such as **\$.invoice.vatBreakdown[0].vatCategoryCode**.
- Include the repeated class as a **type=C** row in the same binding table.

10. Troubleshooting

If an output column is blank, check that the **semantic_path** final segment matches the Structured CSV source column after UpperCamelCase conversion.

If a line-level file has only one row, check that the binding table includes a repeated **type=C** row for **\$.invoice.invoiceLine** with multiplicity **1..*** or **0..***.

If a target filename is unexpected, check the binding CSV filename. The converter derives the target view from names such as **ADS_Invoices_Received_PSV_Binding.csv**.

19. ISO 21378 ADC Invoice Coverage

ISO 21378 ADC Invoice Semantic Binding Coverage

Scope

The Phase 2 semantic binding covers these ISO 21378:2019 Audit Data Collection invoice views:

ADC table	Target view	Binding file
Table 38	SAL Invoice Generated	ISO21378_SAL_Invoice_Generated_CSV_Binding.csv
Table 39	SAL Invoice Generated Details	ISO21378_SAL_Invoice_Generated_Details_CSV_Binding.csv
Table 53	PUR Invoice Received	ISO21378_PUR_Invoice_Received_CSV_Binding.csv
Table 54	PUR Invoice Received Details	ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv

The bindings use the ISO 21378 flat invoice model. Repeated TAX groups are expanded to four numbered column sets. BUSINESS SEGMENT is expanded to five numbered columns.

This scope is complete for Phase 2 of the PoC. The completion criterion is a reviewed target definition, executable binding, generated CSV output for all current Phase 1 inputs, and an explicit classification for fields that cannot be copied directly. Full population of every ISO 21378 field is not a completion criterion because the EN 16931 source model does not contain ERP audit-trail and posting data.

Mapping Coverage

Target view	Fields	Direct	Approximate	Requires transformation	Not available	Bound path
PUR Invoice Received	42	17	5	5	15	22
PUR Invoice Received Details	39	12	1	2	24	13
SAL Invoice Generated	41	17	4	5	15	21
SAL Invoice Generated Details	39	11	1	2	25	12

Bound path is the sum of direct and approximate mappings. A required transformation is not counted as bound because the current semantic binding converter only copies values and does not calculate or parse them.

Data Sufficiency Assessment

The Phase 1 EN 16931 Structured CSV is sufficient for a useful invoice-core projection. It provides invoice identifiers, dates, parties, currency, totals, payment information, VAT breakdowns, invoice lines, quantities, prices, item identifiers, and purchase or sales order references.

It is not sufficient for a complete ISO 21378 ADC delivery. The main gaps are:

- fiscal year and accounting period, which require an accounting calendar;
- normalized payment-term components, which require parsing or dedicated source fields;
- ERP activity records for created, approved, and last modified events;
- invoice lifecycle status;
- general-ledger debit, credit, and tax posting accounts;
- gross line amounts and line tax amounts, which require calculation or allocation;
- separate basic-unit quantities and codes;
- multiple business segment values beyond the single EN 16931 accounting reference;
- multiple tax structures on one invoice line, because EN 16931 defines one VAT classification per line.

Therefore the current bindings are suitable for Phase 2 proof-of-concept generation and mapping-gap analysis, but not for claiming full ISO 21378 ADC completeness.

Test Result

The four bindings were applied to all 10 current Phase 1 Structured CSV invoice files. The run generated 40 target files:

Target view	Files	Total data rows	Minimum rows per file	Maximum rows per file
PUR Invoice Received	10	10	1	1
PUR Invoice Received Details	10	18	1	3
SAL Invoice Generated	10	10	1	1
SAL Invoice Generated Details	10	18	1	3

All non-empty semantic paths in the four bindings were also checked against **EN16931_CIUS_Invoice_LHM.csv**. Every bound path exists in the current UADC LHM.

Detailed **semantic_binding.py** Processing

Phase 2 Semantic Binding: Structured CSV To PSV Or CSV

Script:

```
src/semantic_binding.py
```

Main functions:

```
load_bindings(...)
transform_rows(...)
transform_repeated_group_rows(...)
merge_values(...)
convert_file(...)
```

Input:

- Phase 1 Structured CSV.
- Semantic binding CSV under **specs/bindings/semantic/**.

Output:

- ADS PSV or CSV target file.

Semantic Binding Table

The semantic binding table starts from the target definition columns and adds UADC mapping columns.

```
field_no,field_name,level,flat_file_data_type,length,description,source_document,semantic_path,type,multiplicity
```

Target field rows use **type=A**. Class rows use **type=C**. The class rows are not output columns. They tell the script which semantic classes repeat.

Runtime Objects

read_csv_rows(path) returns:

```
list[dict[str, str]]
```

load_bindings(binding_csv) builds:

```
repeated_classes: dict[str, SemanticClass]
bindings: list[SemanticBinding]
```

SemanticClass holds:

- **semantic_path**;
- **multiplicity**.

Only repeated class rows are kept in **repeated_classes**.

SemanticBinding holds:

- **target_column**;
- **semantic_path**;
- **normalized_semantic_path**;
- **source_column**;
- **repeat_group_path**;
- **repeat_group_column**;
- **repeat_index**.

Source Column Derivation

The source Structured CSV column is derived from the final segment of **semantic_path**.

```
$.invoice.invoiceNumber -> InvoiceNumber
```

```
$.invoice.invoiceLine.invoiceLineIdentifier -> InvoiceLineIdentifier
```

Indexed paths are normalized first.

```
$.invoice.vatBreakdown[0].vatCategoryTaxAmount
-> normalized path: $.invoice.vatBreakdown.vatCategoryTaxAmount
-> source column: VatCategoryTaxAmount
-> repeat group path: $.invoice.vatBreakdown
-> repeat group column: dVatBreakdown
-> repeat index: 0
```

Semantic Dimension Ownership Resolution

Semantic binding resolves fact ownership from the same three binding properties used by syntax binding: **type=C**, **multiplicity**, and **semantic_path**.

1. **load_bindings(...)** reads all rows with **read_csv_rows(...)**.
2. For each **type=C** row, **multiplicity_repeats(...)** decides whether the class defines a repeated row scope. Only repeating classes are placed in **repeated_classes** because a single class does not create a Structured CSV dimension row.
3. **indexed_semantic_path(...)** removes an explicit occurrence selector such as **[0]** while preserving its zero-based **repeat_index**.
4. **source_column_for_path(...)** maps the last semantic path segment to the Structured CSV fact column.
5. **deepest_repeated_class_ancestor(...)** finds the nearest repeated class above each fact.
6. **dimension_column_for_path(...)** converts that class path to its **dXxx** column.
7. **resolve_bindings_for_classes(...)** stores the result in **SemanticBinding.repeat_group_path** and **SemanticBinding.repeat_group_column**.

Thus a fact below a non-repeating child is assigned to the nearest repeating ancestor. A fact below a repeating child is assigned to that child's own dimension from its first occurrence. This ownership controls which sparse Structured CSV rows may supply the fact; it does not by itself copy parent facts onto a Structured CSV child row.

Row Scope Selection

resolve_bindings_for_classes(bindings, repeated_classes) assigns row-scope information to each **SemanticBinding**. For non-indexed paths, it searches the deepest repeated **C** ancestor in the binding table.

Example:

```
semantic_path: $.invoice.invoiceLine.invoiceLineNetAmount
repeated class: $.invoice.invoiceLine
```

```
repeat_group_column: dInvoiceLine
```

row_scope_group(bindings) determines whether the target file emits one row per repeated occurrence.

Rules:

1. Indexed bindings are ignored as row-scope candidates.
2. Non-indexed bindings with repeated class ancestors are candidates.
3. The deepest repeated class wins.

Thus **Invoices_Received_Lines** emits one row per **dInvoiceLine**, while **Invoices_Received** stays at invoice level even though it may contain indexed VAT breakdown columns.

Transformation Without Repeated Row Scope

If **row_scope_group(...)** returns no scope, **transform_rows(...)** creates invoice-level target rows.

Working variables:

- **output_rows:** list[dict[str, str]];
- **current_row:** dict[str, str] | None;
- **repeat_counts:** dict[str, int];
- **key_binding:** SemanticBinding.

The script scans Structured CSV rows. **row_has_bound_data(...)** skips irrelevant sparse rows. **row_repeat_indices(...)** assigns occurrence numbers for indexed groups. **merge_values(...)** copies non-empty values into the current target row.

Transformation With Repeated Row Scope

If **row_scope_group(...)** returns a scope, **transform_repeated_group_rows(...)** emits one target row per repeated occurrence.

Example:

```
scope_path=$.invoice.invoiceLine
scope_column=dInvoiceLine
```

Working variables:

- **parent_context:** dict[str, str]: invoice-level values.
- **current_group_row:** dict[str, str] | None: current repeated output row.
- **current_scope_value:** str: current **dXxx** occurrence value.
- **parent_bindings:** list[SemanticBinding].
- **scoped_bindings:** list[SemanticBinding].

Rows with **dInvoice** and no selected **scope_column** fill **parent_context**. Rows with the selected **scope_column** start or continue a repeated output row. When the **scope_column** value changes, the current target row is appended to **output_rows** and a new row starts from **parent_context**.

The first populated repeated dimension value therefore starts the first independent target record. Parent values are copied from **parent_context** into that flat target record; they are not expected to be duplicated on the input Structured CSV child row. This distinction preserves the hierarchical input rule while still producing self-contained ADS line records.

Value Merge Rules

merge_values(target_row, source_row, bindings, repeat_indices) is the only function that writes values into a target row.

Rules:

1. Skip bindings without **source_column**.
2. If **repeat_index** exists, require the current occurrence number to match.
3. Read **source_row[source_column]**.
4. Use **default_value** if the source is blank.
5. Write to **target_row[target_column]** only if the target cell is still blank.

This first-non-empty rule is important because Structured CSV rows are sparse and hierarchical.

Semantic Binding Output And Orchestration

1. **row_has_bound_data(...)** ignores sparse rows irrelevant to the selected target view.
2. **row_scope_group(...)** selects invoice-level transformation or the deepest non-indexed repeated target scope.
3. **row_repeat_indices(...)** and **repeated_group_present(...)** maintain zero-based indices used by explicitly indexed semantic bindings.
4. **transform_rows(...)** dispatches to invoice-level merging or **transform_repeated_group_rows(...)**.
5. **write_target_file(...)** writes binding-order columns with the selected delimiter.
6. **output_format_options(...)** resolves PSV or CSV defaults and command-line overrides.
7. **output_path(...)** and **binding_target_stem(...)** derive the target file location and name.
8. **convert_file(...)** performs one input-file conversion, while **main(...)** expands directory inputs with **input_files(...)**.